Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 07

Prof. Claudio Benossi

1.
Unidade

Vetores, Matrizes, TAD e Alocação Dinâmica de Memória

Listas e Pilhas (Stacks)

Lista Encadeada

Uma diferença marcante também da lista encadeada da lista normal é que precisamos guardar além da referencia do primeiro elemento na lista encadeada precisamos guardar também a referencia do último elemento, porém precisamos percorrer toda a lista.

Listas e Pilhas (Stacks)

Exemplo de estrutura Estática:

Vetor

Elemento 1	Elemento 2	Elemento 3	null	null
Posição 0	Posição 1	Posição 2	Posição 3	Posição 4

Exemplo de estrutura Dinâmica:

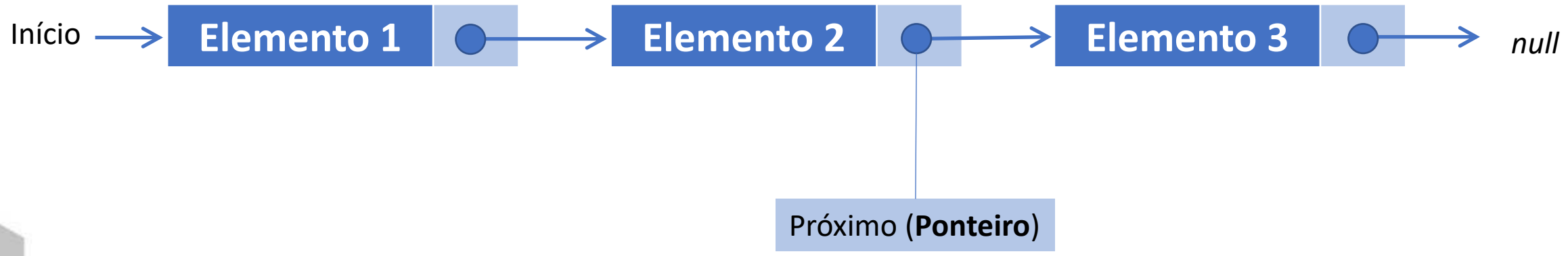
Elemento 3

Elemento 1

Elemento 2

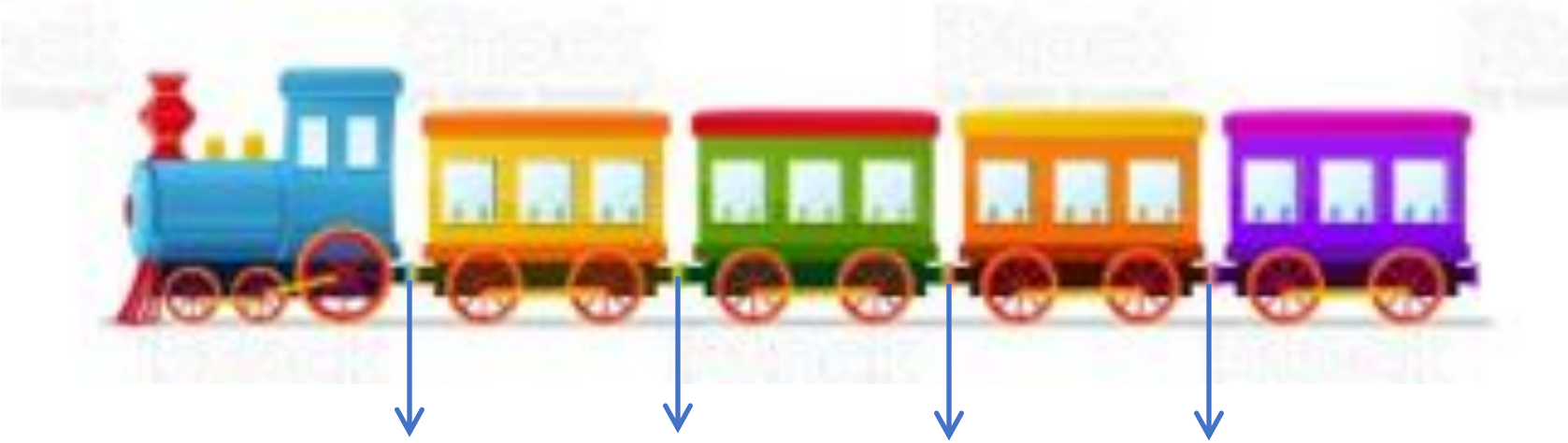
Listas e Pilhas (Stacks)

Com uma lista encadeada vamos alocar o espaço na memoria sobre demanda e depois precisamos ligar esses elementos que a princípio estão espalhados, por isso também chamamos de lista Ligada.



Ponteiros

Fazendo uma analogia:



Você só pode entrar nesse trem pela porta que fica no inicio do trem e para ir até o último vagão você terá que passar por todos e cada pino que liga os vagões são vistos como os ponteiros (**elo de ligação**).

Atividade:

Para facilitar o entendimento vamos criar em exemplo onde:

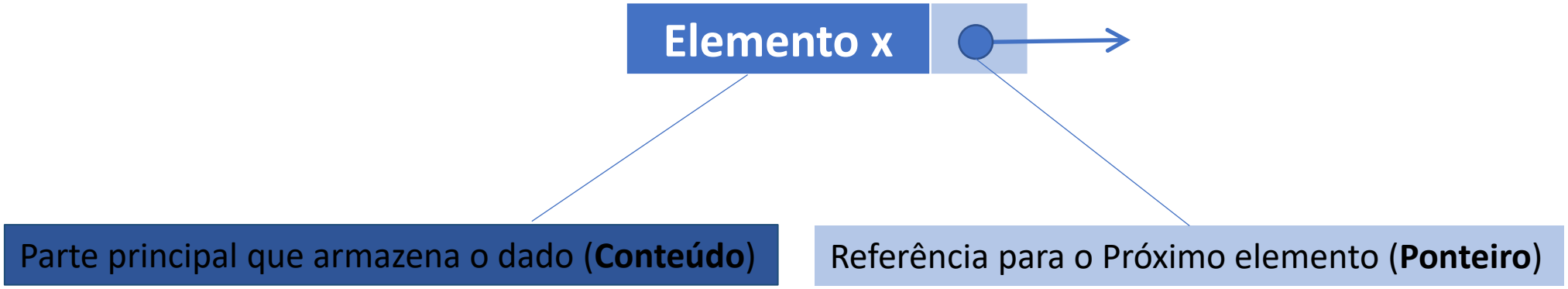
- Podemos ciar uma lista de nó como ListaEncadeada;
- Inserir elemento no início;
- Verificar a quantidade de nós (tamanho);
- Inserir elemento no final da lista;
- Percorrer a lista (visualizar);
- Limpar a Lista;
- Pesquisar um nó ou posição especifica;
- Inserir em posição específica.
- Remover do início;
- Remover do final da lista;
- Remover de qualquer posição;
- Inserir de forma ordenada;
- API Java LinkedList;

Valendo Nota !!!

Lista Encadeada

Para esse exemplo vamos criar um novo projeto chamado de **Lista** e dentro dele vamos criar nossas classes (**ListaEncadeada e No – Além da classe Lista que podemos usar para testar)**, lembrando que vamos ter:

Nó



Lista Encadeada

Código da classe **ListaEncadeada.java**

Lista.java | **ListaEncadeada.java** | No.java

Código-Fonte | Histórico

```
1   package lista;
2
3   public class ListaEncadeada<Tipo> {
4       private No inicio;
5       private No fim;
6       private int tamanho;
7
8   }
```

Atributos

Lista Encadeada

Código da classe **ListaEncadeada.java**

Precisamos criar métodos de acesso (Get e Set) e construtor, além de métodos para manipular os dados e realizar a sua exibição, então vamos lá ....

Mão na massa ...

Lista Encadeada

Lista.java | **ListaEncadeada.java** | No.java

Código-Fonte | Histórico

```java
package lista;

public class ListaEncadeada<Tipo> {
    private No inicio;
    private No fim;
    private int tamanho;

    // Método Construtor
    public ListaEncadeada() {
        this.tamanho = 0;
    }

}
```

Lista Encadeada

Precisamos ter o controle de quantos **Nós** temos em nossa estrutura, porque se houver uma quantidade muito grande em nossa estrutura não seria muito indicado percorrer toda a lista para saber a quantidade de elementos armazenados, nesse caso isso levaria a um custo muito alto no ponto de vista computacional; e como alternativa é recomentado criar em nossa estrutura um nova variável para fazer esse controle, no caso, tamanho.

Lista Encadeada

Após criar o atributo tamanho na classe **ListaEncadeada**, vamos criar métodos de acesso **getTamanho()** para obter os dados, e **setTamanho()**;

Também será necessário alterar o método **adiciona()** para realizar esse controle:

Lista Encadeada

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class ListaEncadeada<Tipo> {
    private No inicio;
    private No fim;
    private int tamanho;

    // Método Construtor
    public ListaEncadeada() {
        this.tamanho = 0;
    }

   // Métodos de Acesso
    public No getInicio() {
        return inicio;
    }

    public void setInicio(No inicio) {
        this.inicio = inicio;
    }

    public No getFim() {
        return fim;
    }

    public void setFim(No fim) {
        this.fim = fim;
    }

}
```

Lista Encadeada

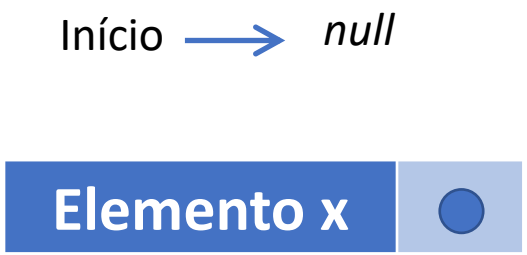
Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
1   package lista;
2
3   public class ListaEncadeada<Tipo> {
4       private No inicio;
5       private No fim;
6       private int tamanho;
7
8       // Método Construtor
9       public ListaEncadeada() {...3 linhas }
12
13      // Métodos de Acesso
14      public No getInicio() {...3 linhas }
17
18      public void setInicio(No inicio) {...3 linhas }
21
22      public No getFim() {...3 linhas }
25
26      public void setFim(No fim) {...3 linhas }
29
30      public int getTamanho() {...3 linhas }
33
34      public void setTamanho(int tamanho) {...3 linhas }
37
38      // Método de Exibição
39      @Override
        public String toString() {
41          return "ListaEncadeada{" + "inicio=" + inicio + '}';
42      }
43
44  }
```

Lista Encadeada

Lembrando quando vamos adicionar um conteúdo no início da nossa lista encadeada o início aponta para null já que não temo nada ainda em nossa lista:



Lista Encadeada

Código da classe **ListaEncadeada.java**

Agora vamos criar os métodos para manipular os dados...

Lista.java | **ListaEncadeada.java** | No.java

Código-Fonte | Histórico

```
44          // Método para adicionar elemento
45              public void adiciona(Tipo elemento){
                No<Tipo> celula = new No<Tipo>(elemento);
47              if (this.inicio == null && this.fim == null){
48                  this.inicio = celula;
49                  this.fim = celula;
50              } else {
51                  this.fim.setProximo(celula);
52                  this.fim = celula;
53              }
54              this.tamanho++;
55          }
56
57      }
```

Lista Encadeada

Agora vamos criar a nossa classe **No.java**:

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
1   package lista;
2
3   public class No<Tipo> {
        private Tipo elemento;
        private No<Tipo> proximo;
6
7       public No(Tipo elemento) {
8           this.elemento = elemento;
9           this.proximo = null;
10      }
11
12  }
```

Precisamos criar métodos como o construtor, métodos de acesso e de exibição

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class No<Tipo> {
    private Tipo elemento;
    private No<Tipo> proximo;

    public No(Tipo elemento) {
        this.elemento = elemento;
        this.proximo = null;
    }

    public No(Tipo elemento, No<Tipo> proximo) {
        this.elemento = elemento;
        this.proximo = proximo;
    }

    public Tipo getElemento() {
        return elemento;
    }

    public void setElemento(Tipo elemento) {
        this.elemento = elemento;
    }

    public No<Tipo> getProximo() {
        return proximo;
    }

    public void setProximo(No<Tipo> proximo) {
        this.proximo = proximo;
    }

    @Override
    public String toString() {
        return "No{" + "elemento=" + elemento + ", proximo=" + proximo + '}';
    }

}
```

Lista Encadeada

OK agora podemos realizar alguns testes para verificar a nossa estrutura de dados, para isso vamos na nossa classe **Lista.Java**:

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        System.out.println("Tamanho da Lista = " + lista.getTamanho());

        lista.adiciona("Claudio");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        lista.adiciona("Camila");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());
    }
}
```

Lista Encadeada

Resultado esperado:

Saída - Lista (run)

```
run:
Tamanho da Lista = 0
ListaEncadeada{inicio=No{elemento=Claudio, proximo=null}}
Tamanho da Lista = 1
Início da Lista = Claudio
Fim da Lista = Claudio
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=null}}}
Tamanho da Lista = 2
Início da Lista = Claudio
Fim da Lista = Camila
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```

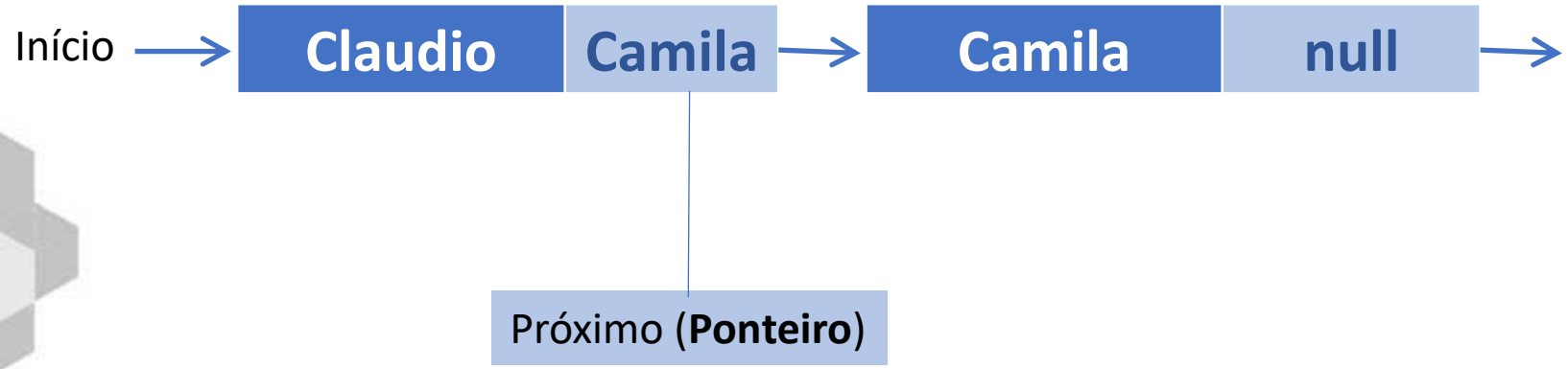
Lista Encadeada

Fazendo uma comparação:

Vetor

Claudio	Camila	null	null	null
Posição 0	Posição 1	Posição 2	Posição 3	Posição 4

Lista Encadeada:



Lista Encadeada

Até o momento temos somente dois nomes em nossa lista, porém vamos imaginar que podemos ter vários nomes nesta lista, então precisamos criar um método na nossa classe **ListaEncadeada.java** para buscar um elemento, lembrando que não temos index nesta estrutura.

Lista Encadeada

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```
57          // Método para buscar o Nó
58              public No get(int posicao){
59                  No atual = this.inicio;
60                  for(int i = 0; i < posicao; i++){
61                      if(atual.getProximo() != null){
62                          atual = atual.getProximo();
63                      }
64                  }
65                  return atual;
66              }
67
68      }
```

Lista Encadeada

Vamos testar:

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        lista.adiciona("Claudio");
        lista.adiciona("Camila");
        lista.adiciona("Miguel");
        lista.adiciona("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        System.out.println("Elemento na posição 2 = " + lista.get(2).getElemento());
    }
}
```

Lista Encadeada

Resultado:

Saída - Lista (run)

```
run:
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}}
Tamanho da Lista = 4
Início da Lista = Claudio
Fim da Lista = Elis
Elemento na posição 2 = Miguel
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Lista Encadeada

OK, nós inserimos dados em nossa lista, também verificamos o seu tamanho e até buscamos valores em posições específicas (***isso não é muito usado em lista encadeada***), agora vamos remover um elemento desta lista encadeada, através do método remover().

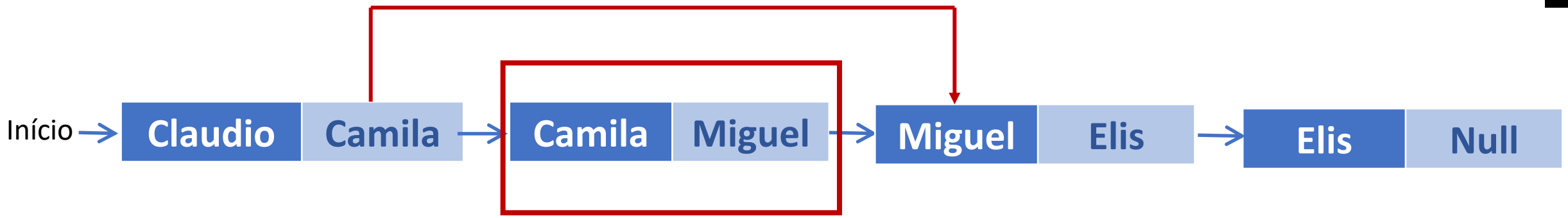
Lista Encadeada

Lembrando que eu tenho 3 (três) possibilidades, quando for fazer a remoção:

Lista Encadeada

Lembrando que eu tenho 3 (três) possibilidades, quando for fazer a remoção:

1ª - Remover um elemento no meio

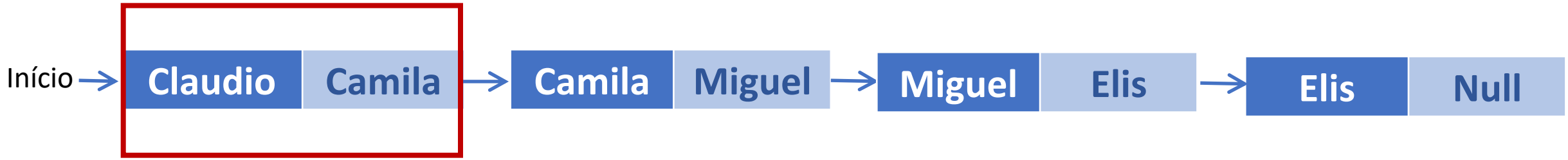


O ponteiro do elemento Claudio deve passar a ser Miguel

Lista Encadeada

Lembrando que eu tenho 3 (três) possibilidades, quando for fazer a remoção:

2ª - Remover um elemento no início

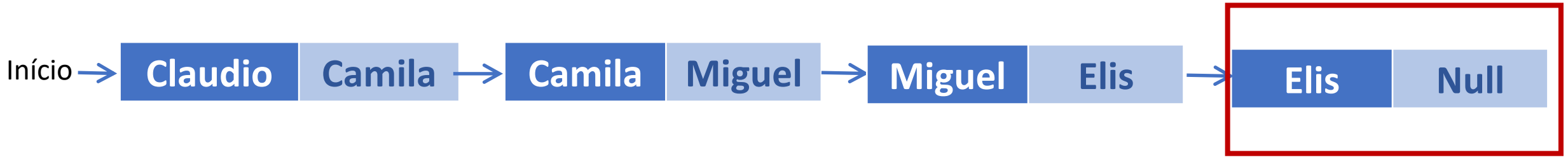


Neste caso o início passaria a ser Camila

Lista Encadeada

Lembrando que eu tenho 3 (três) possibilidades, quando for fazer a remoção:

3ª - Remover um elemento no fim



Neste caso o ponteiro do Miguel passaria a ser Null

Lista Encadeada

1ª - Remover um elemento no meio

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
68          // Método para remover Nó
69          public void remover(Tipo elemento){
70              No anterior = null;
71              No atual = this.inicio;
72              for (int i = 0; i < this.getTamanho(); i++){
73                  if (atual.getElemento().equals(elemento)){
                        anterior.setProximo(atual.getProximo());
                        atual = null;
76                      this.tamanho--;
77                      break;
78                  }
79                  anterior = atual;
80                  atual = atual.getProximo();
81              }
82          }
83      }
```

Lista

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        lista.adiciona("Claudio");
        lista.adiciona("Camila");
        lista.adiciona("Miguel");
        lista.adiciona("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        lista.remover("Camila");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());
    }
}
```

Lista Encadeada

Resultado:

Saída - Lista (run)

```
run:
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}}
Tamanho da Lista = 4
Início da Lista = Claudio
Fim da Lista = Elis
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}
Tamanho da Lista = 3
Início da Lista = Claudio
Fim da Lista = Elis
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Lista Encadeada

Vamos tratar as outras duas possibilidades, quando for fazer a remoção:

2ª - Remover um elemento no início

3ª - Remover um elemento no fim

Lista

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
68          // Método para remover Nó
69          public void remover(Tipo elemento){
70              No anterior = null;
71              No atual = this.inicio;
72              for (int i = 0; i < this.getTamanho(); i++){
73                  if (atual.getElemento().equals(elemento)){
74                      if (this.tamanho == 1){
75                          this.inicio = null;
76                          this.fim = null;
77                      } else if(atual == inicio){
78                          this.inicio = atual.getProximo();
79                          atual.setProximo(null);
80                      } else if (atual == fim){
81                         this.fim = anterior;
                           anterior.setProximo(null);
83                      } else
                            anterior.setProximo(atual.getProximo());
                            atual = null;
86                          this.tamanho--;
87                          break;
88                  }
89                  anterior = atual;
90                  atual = atual.getProximo();
91              }
92          }
93      }
```

Lis

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        lista.adiciona("Claudio");
        lista.adiciona("Camila");
        lista.adiciona("Miguel");
        lista.adiciona("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        lista.remover("Claudio");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());
    }
}
```

Lista Encadeada

Resultado:

Saída - Lista (run)

```
run:
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}}
Tamanho da Lista = 4
Início da Lista = Claudio
Fim da Lista = Elis
ListaEncadeada{inicio=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}
Tamanho da Lista = 3
Início da Lista = Camila
Fim da Lista = Elis
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Lista

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        lista.adiciona("Claudio");
        lista.adiciona("Camila");
        lista.adiciona("Miguel");
        lista.adiciona("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        lista.remover("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());
    }
}
```

Lista Encadeada

Resultado:

Saída - Lista (run)

```
run:
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}}
Tamanho da Lista = 4
Início da Lista = Claudio
Fim da Lista = Elis
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=null}}}}
Tamanho da Lista = 3
Início da Lista = Claudio
Fim da Lista = Miguel
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Lista

Lista.java | ListaEncadeada.java | No.java

Código-Fonte | Histórico

```java
package lista;

public class Lista {

    public static void main(String[] args) {
        ListaEncadeada<String> lista = new ListaEncadeada<>();

        lista.adiciona("Claudio");
        lista.adiciona("Camila");
        lista.adiciona("Miguel");
        lista.adiciona("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());

        lista.remover("Claudio");
        lista.remover("Camila");
        lista.remover("Miguel");
        lista.remover("Elis");

        System.out.println(lista);
        System.out.println("Tamanho da Lista = " + lista.getTamanho());
        System.out.println("Início da Lista = " + lista.getInicio().getElemento());
        System.out.println("Fim da Lista = " + lista.getFim().getElemento());
    }
}
```

Lista Encadeada

Resultado:

Saída - Lista (run)

```
run:
ListaEncadeada{inicio=No{elemento=Claudio, proximo=No{elemento=Camila, proximo=No{elemento=Miguel, proximo=No{elemento=Elis, proximo=null}}}}}
Tamanho da Lista = 4
Início da Lista = Claudio
Fim da Lista = Elis
ListaEncadeada{inicio=null}
Tamanho da Lista = 0
Exception in thread "main" java.lang.NullPointerException
        at lista.Lista.main(Lista.java:25)
C:\Users\claud\AppData\Local\NetBeans\Cache\8.2rc\executor-snippets\run.xml:53: Java returned: 1
FALHA NA CONSTRUÇÃO (tempo total: 0 segundos)
```

Lista Encadeada

Ok, conseguimos inserir, consultar, e remover elementos de nossa lista encadeada.

A classe **LinkedList** é uma implementação da interface List.

Esta classe implementa boa parte das operações de uma lista e permite a inserção de qualquer tipo de elemento (incluindo null). ... *Não é raro ver programadores usando a classe LinkedList como uma pilha ou fila*.

Está Classe já vem com uma série de métodos prontos, saiba mais...

https://www.tutorialcup.com/pt/Java/lista-vinculada-em-java.htm

"Todos os nossos sonhos podem-se realizar,
se tivermos a coragem de persegui-los”

Walt Disney

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br